Part II (5 points):

In unit2, we discussed the use of threads for parallel computing and, in unit3, we discussed different synchronization techniques to avoid race conditions between cooperating processes/threads. In this question, you are designing a multi-threaded traffic intersection simulation where cars approach a four-way intersection. The main goal is to ensure safe and deadlock-free traffic flow at the intersection.

Your simulator is structured as follows:

- Each car is represented by a thread that approaches the intersection from one of the four directions (North, South, East, West).
- Each car is a separate thread, randomly arriving at the intersection.
- Once a car enters the intersection, it should sleep for a short time (simulating crossing time) before exiting.
- Each car must wait for the right conditions to pass safely and only one car passes through the intersection at a time.
- The intersection should be protected using synchronization mechanisms to prevent collisions ○ Each car must acquire a lock before proceeding to prevent collisions.
- Deadlocks (e.g., cars are waiting indefinitely due to circular waiting) must be avoided. You can utilize one of the following techniques:
    - Lock ordering (acquire locks in a consistent order).
    - Lock with timeout (retry if locks cannot be acquired).
    - Central Traffic controller using semaphores (only allow a limited number of cars at once).

Running your simulator:

With your simulator running in an infinite loop, display the current status of this intersection in the format you find appropriate. Attached below is ***an example*** of such status.

```
[Car 1] approaching from North.
[Car 2] approaching from East.
[Car 3] approaching from South.
[Car 4] approaching from West.

[Car 1] from North is waiting at the intersection.
[Car 2] from East is waiting at the intersection.
[Car 3] from South is waiting at the intersection.
[Car 4] from West is waiting at the intersection.

[Car 1] from North is crossing the intersection.
[Car 1] from North has exited.

[Car 2] from East is crossing the intersection.
[Car 2] from East has exited.

……
```

```java
import java.util.concurrent.Semaphore;
import java.util.Random;
import java.util.ArrayList;
import java.util.List;
import java.util.Collections;

public class CarProblem {

    public static  String[] directions = {"North", "South", "East", "West"};
    public static  Semaphore s = new Semaphore(1);
    public static  Random rand = new Random();

    private static int carCount = 1;

    public static void main(String[] args)
    {

        int scenario = 1;                                           // Used this to help identify cars currently at the
intersection
        while (true)                                                // infinite loop
        {

            List<Integer> cars = new ArrayList<>();                 //Lists car numbers
            List<String> carDirections = new ArrayList<>();         //Lists the direction for the car
            List<Integer> arr = new ArrayList<>();              //List to help create a random order for the cars to
cross the intersection

            System.out.println("\nScenario " + scenario + "\n");    // Displays the current Scenario count

            for (String direct : directions)                        //This helped with getting different number of cars
to approach the intersection
            {
                                                // If number that is picked at random is less than 50 then it will add the car
                if (rand.nextInt(100) < 50) {
                    cars.add(carCount++);                           //The car is assigned a number based on the count
                    carDirections.add(direct);                      //Assigns the car with the direction its coming from
                }
            }

            if (cars.isEmpty())                                     // Moves to the next scenario if no cars are in the
intersection otherwise it continues
            {
                System.out.println("\n(No cars currently... Intersection is empty)\n");
                scenario++;                                         // Increment and move to next scenario
                continue;
            }
```

```
            System.out.println();                                // Prints the cars approaching the intersection

            for (int i = 0; i < cars.size(); i++)
            {

                System.out.println("[Car " + cars.get(i) + "] approaching from " + carDirections.get(i) + ".");
            }

            System.out.println();                               // Print cars waiting

            for (int i = 0; i < cars.size(); i++)
            {

                System.out.println("[Car " + cars.get(i) + "] from " + carDirections.get(i) + " is waiting at the
intersection.");
            }


            for (int i = 0; i < cars.size(); i++)
            {

                arr.add(i);                                  //Adds onto the index based on the size of the cars array
            }

            Collections.shuffle(arr);                      //Helps shuffle array to allow random car crossing because I
had an issue with the same car crossing the intersection

            Thread[] threads = new Thread[cars.size()];    //Thread array


            for (int i = 0; i < arr.size(); i++)
            {

                int index = arr.get(i);                         // Gets the random index from the array
                int carNum = cars.get(index);                   // Gets car num
                String direction = carDirections.get(index);    // Gets the direction for the car

                threads[index] = new Thread(new Runnable()
                {

                    public void run()
                    {

                        try
                        {

                            s.acquire();                         // Entering CRITICAL SECTION acquiring semaphore lock
```

```java
                            System.out.println();
                            System.out.println("[Car " + carNum + "] from " + direction + " is crossing the
intersection.");

                            Thread.sleep(rand.nextInt(3000));          // car crossing the intersection

                            System.out.println("[Car " + carNum + "] from " + direction + " has exited.\n");

                        } catch (InterruptedException e)
                        {
                            Thread.currentThread().interrupt();
                        }

                        s.release();                                    // Exit CRITICAL SECTION release semaphore lock
                    }
                });

                threads[index].start();                                   // Starts the threads at the index
            }

            for (int i = 0; i < threads.length; i++)
            {
                try {
                    if (threads[i] != null)
                    {
                        threads[i].join();              // waits for all car threads to finish before the loop starts
over
                    }
                } catch (InterruptedException e)
                {
                    Thread.currentThread().interrupt();
                }
            }

            scenario++;                                 // Increment and move to next scenario
        }
    }
}
```

```
Scenario 1

[Car 1] approaching from North.
[Car 2] approaching from South.
[Car 3] approaching from East.
[Car 4] approaching from West.

[Car 1] from North is waiting at the intersection.
[Car 2] from South is waiting at the intersection.
[Car 3] from East is waiting at the intersection.
[Car 4] from West is waiting at the intersection.

[Car 3] from East is crossing the intersection.
[Car 3] from East has exited.

[Car 2] from South is crossing the intersection.
[Car 2] from South has exited.

[Car 1] from North is crossing the intersection.
[Car 1] from North has exited.

[Car 4] from West is crossing the intersection.
[Car 4] from West has exited.

Scenario 2

[Car 5] approaching from East.

[Car 5] from East is waiting at the intersection.

[Car 5] from East is crossing the intersection.
[Car 5] from East has exited.

Scenario 3

[Car 6] approaching from North.
[Car 7] approaching from South.
[Car 8] approaching from East.
[Car 9] approaching from West.
```

```
Scenario 3

[Car 6] approaching from North.
[Car 7] approaching from South.
[Car 8] approaching from East.
[Car 9] approaching from West.

[Car 6] from North is waiting at the intersection.
[Car 7] from South is waiting at the intersection.
[Car 8] from East is waiting at the intersection.
[Car 9] from West is waiting at the intersection.

[Car 9] from West is crossing the intersection.
[Car 9] from West has exited.

[Car 7] from South is crossing the intersection.
[Car 7] from South has exited.

[Car 8] from East is crossing the intersection.
[Car 8] from East has exited.

[Car 6] from North is crossing the intersection.
[Car 6] from North has exited.

Scenario 4

[Car 10] approaching from North.
[Car 11] approaching from South.

[Car 10] from North is waiting at the intersection.
[Car 11] from South is waiting at the intersection.

[Car 11] from South is crossing the intersection.
[Car 11] from South has exited.

[Car 10] from North is crossing the intersection.
[Car 10] from North has exited.

Scenario 5
```

```
[Car 72] from South is crossing the intersection.
[Car 72] from South has exited.


[Car 73] from West is crossing the intersection.
[Car 73] from West has exited.


Scenario 37


[Car 74] approaching from North.
[Car 75] approaching from South.
[Car 76] approaching from West.

[Car 74] from North is waiting at the intersection.
[Car 75] from South is waiting at the intersection.
[Car 76] from West is waiting at the intersection.

[Car 75] from South is crossing the intersection.
[Car 75] from South has exited.


[Car 74] from North is crossing the intersection.
[Car 74] from North has exited.


[Car 76] from West is crossing the intersection.
[Car 76] from West has exited.


Scenario 38


(No cars currently... Intersection is empty)


Scenario 39


[Car 77] approaching from East.

[Car 77] from East is waiting at the intersection.

[Car 77] from East is crossing the intersection.
[Car 77] from East has exited.


Scenario 40
```